

Computationalism, Connectionism, and the Philosophy of Mind

Brian P. McLaughlin

Introduction

The central questions of the philosophy of mind are the nature of mental phenomena, and how mental phenomena fit into the causal structure of reality. The computational theory of mind aims to answer these questions. The central tenet of the theory is that a mind is a computer. According to the theory, mental states and events enter into causal relations via operations of the computer. The main aim of the theory is to say what kind of computer – what kind of computational mechanism – a mind is. The answer is still unknown. Pursuing it is the main research program of the theory.

In the most general sense, a computer is, roughly, a system of structures functionally organized in such a way as to be able to compute. The structures, their functional organization, and the basic modes of operation of the system when it computes comprise the functional architecture of the computer. The two tasks of the computational theory of mind are: (1) to identify the functional architecture of the computing system that grounds our mental abilities

and (2) to explain how those abilities are exercised via operations of the system. The tasks are related. The explanation of how operations of the system constitute exercises of our mental abilities will justify the claim that our possession of those abilities consists in our being at least partly constituted by the system.

Computationalists hold that the functional architecture of the computing system that grounds our mental abilities resides in our brains. There is, however, no consensus as to what even the general character of that architecture is. The symbols-system paradigm and the connectionist paradigm are the two dominant research paradigms within the computational theory of mind. They differ primarily in what kind of computer the mind is assumed to be, and thus in the kinds of functional architectures explored. The symbol-system paradigm presupposes that the mind is a kind of automatic formal system, while the connectionist paradigm presupposes that it is a system of connectionist networks. These paradigms will be discussed in due course. First, however, further general discussion of the computational theory of mind is in order.

1 The Computational Theory of Mind

Cognitive functions

Having a mind consists in having mental abilities such as, for example, the ability to think. According to the computational theory of mind, the exercise of mental abilities consists in the computation of certain functions – cognitive functions. The theory is thus concerned with what functions are cognitive functions and how they are computed.

Functions are relations, but not every relation is a function. A function is a one–one or many–one relation. Functions have arguments and values. The set of arguments of the function is its domain and the set of values is its range. For each argument in the domain, there is a unique value in the range. Functions are thus extensionally characterized as sets of ordered pairs, the first element of each ordered pair being an argument of the function, the second, the unique value of the function for that argument. Addition, multiplication, and division, for example, are mathematical functions that take order pairs of numbers as arguments and have numbers as values. A computable function is one that can be computed. Not all functions are computable. A function is computable if and only if there is an algorithm for finding a value of the function for any argument of the function. An algorithm is a kind of procedure, a way of getting something done; but not every procedure is an algorithm. An algorithm is an effective procedure, a sure-fire way of getting something done in a finite number of steps. It is sure-fire in that if each step is followed exactly, success is guaranteed. Each step of an algorithm is, moreover, easy in the sense that it requires no ingenuity or intelligence to carry it out, and thus “a mere mechanism” could execute it.

To compute is to compute a function, and to compute a function is to execute an algorithm. More than one algorithm can be used to compute a function. For example, the multiplication function – understood extensionally as a set of ordered pairs – can be computed using either the partial-product method taught in elementary

school or by a series of additions. In fact, for any computable function, there will be infinitely many algorithms for computing it. The reason is trivial: we can always add intermediate steps to an algorithm for computing the function that contribute nothing to its computation.

How a function is computed will depend on the functional architecture of the machine (the computing system) that computes it, including the representational code used in the machine. Thus, for example, the algorithms for doing addition in base 10 differ from those for doing addition in binary notation, which uses only 1's and 0's. Likewise, the algorithms for doing addition, multiplication, and division using Roman numerals are different from those for computing these functions in the Arabic numerals we were taught in grade school.

These methods of calculating are algorithms for manipulating numerals (symbols that represent numbers), as are the methods for calculation using other kinds of numeral systems. However, the symbols manipulated by an algorithm can be symbols for anything that can be represented: persons, places, things, etc. And since a symbol can purport to represent something yet fail, an algorithm can manipulate symbols that purport to represent something yet fail.

Symbols have formal as well as semantic properties. The formal properties of symbols are intrinsic nonsemantic properties, such as shape or syntactic structure. Symbolic algorithms operate on symbols in virtue of certain of their formal properties, rather than their semantic ones. Thus, for example, the mathematical operations on Arabic numerals are defined over the shapes of the numerals, and likewise for the mathematical operations on Roman numerals. The beauty of symbolic algorithms is that they can involve the manipulation of symbols in ways that makes sense given what the symbols represent. Thus, while the algorithms for adding Arabic numerals are different from those for adding Roman numerals, the symbol transitions that occur during the execution of the algorithms in question can be systematically interpreted as adding the numbers represented.

Not all algorithms are symbol manipulation algorithms. An algorithm for getting all of the sides of a Rubik's Cube to match in color, for

example, involves the manipulation of the squares of the Cube, not the manipulation of symbols representing them (Cummins & Schwarz 1991). The computational theory of mind is concerned with the algorithms by which we compute cognitive functions. As will become apparent in due course, while the symbol-system paradigm appeals to symbol manipulation algorithms to explain how cognitive functions are computed, the connectionist computational paradigm often appeals to non-symbolic algorithms to do so.

When we execute the partial-product algorithm using pencil and paper, for instance, each step of the process is guided by our intentions and decisions; likewise, when we execute by hand an algorithm for getting all of the sides of a Rubik's Cube to match in color. Computationalists maintain that the execution of basic cognitive algorithms (ones not executed by means of the execution of other cognitive algorithms) can be described and explained without invoking intelligent guidance for any step, and so without having to invoke "homunculi" – little intentional agents. This ultimate elimination of homunculi is essential since computationalism aims to reductively explain intentional abilities. (See the discussion of homunculi in Sterelny 1990.)

To see how an algorithm can be executed completely mindlessly, consider electronic circuits that function as truth-functional gates. For example, an AND-gate has two input wires and one output wire. When it receives current along both of its input wires it closes, thereby sending current along its output wire. When it receives current from only one input wire or does not receive current from either input wire, it remains open and no current is sent along its output wire. This circuit is called an "AND-gate" because using *T* to represent current being sent current along a wire and *F* to represent the absence of current being sent along a wire, we can construct a machine table representing how this circuit works that will exactly resemble the truth-table for conjunction, the truth function expressed by central uses of "and" in English. An AND-gate can thus be interpreted as computing the truth-function conjunction. Indeed for any truth table, no matter how complex, there is a formally equivalent electronic circuit design, and the converse. The

electronic circuitry of such a device acts completely mindlessly.

The computational, algorithmic, and implementational levels

David Marr (1982) perspicuously distinguished three levels of analysis for addressing a computational problem. At the computational level of analysis, one specifies *what* cognitive function is being computed. At the algorithmic level, one describes *how* the function is being computed, the algorithm used to compute it. And at the implementational level, one describes how the steps of that algorithm are implemented, that is, the underlying mechanism by whose operations they are taken.

The three levels are relative to a computational problem. What is at the implementational level relative to one computational problem can be at either the computational or algorithmic level relative to another. The reason is that the implementation of a step of an algorithm might itself be characterized as the computation of a function, one executed by a different algorithm, whose steps are themselves implemented somehow; and each step of the latter algorithm might itself be characterized as the computation of a function, and so on, and so forth. Different algorithms are executed at different scales in the brain. It is an unanswered empirical question whether all cognitive functions are executed by algorithms involving elements at the same scale (Churchland & Sjenowski 1993).

Marr (1982) suggested that computational level analysis could be carried out largely independently of algorithmic level analysis, and the latter largely independently of implementational analysis. Now it is certainly true that the computational level underdetermines the algorithmic level: infinitely many algorithms can compute the same function. Moreover, the algorithmic level in turn underdetermines the implementational level: a given algorithm can be implemented in infinitely many possible ways. Nonetheless, the computationalist's concern is not merely to discover what cognitive functions are computed, but also to discover the algorithms used to compute them. Since what algorithms a machine can

execute will depend on its functional architecture, attention to likely modes of implementation can help in discovering those algorithms.

What algorithms a machine can execute supervenes on the functional architecture of the machine. (*A*-properties supervene on *B*-properties just in case the two things cannot differ in respect of *A*-properties without differing in respect of *B*-properties.) Thus, there cannot be a difference in what algorithms two machines can compute without a difference in their functional architectures. It follows that two machines with exactly the same functional architecture can execute exactly the same algorithms. Similarly, what functions a machine can compute will supervene on what algorithms it can execute, and so two machines that can execute exactly the same algorithms can compute exactly the same functions. Since supervenience is transitive, what functions a machine can compute will supervene on the functional architecture of a machine. As indicated above, however, functional architecture does not supervene on computational power. Machines with very different architectures can have exactly the same computational power.

Turing machines and Turing computability

In his early machine-state psychofunctionalism, Hilary Putnam (1975) claimed that we are finite state automata, by which he meant that we are Turing machines with finite tapes. The idea that our minds have the functional architecture of a Turing machine with a finite tape is not to be taken seriously. No computationalist thinks the mind has a Turing machine architecture. One of Putnam's essential points, however, was that we have the computational capacities of a Turing machine with a finite tape, in that our cognitive functions are computable by a Turing machine. According to the computational theory of mind, all cognitive functions can be computed by algorithms – effective procedures. According to the Church–Turing thesis, every function that can be computed by an algorithm can be computed by a Turing machine. If the Church–Turing thesis is correct, then the computational theory of mind is committed to the thesis that

every cognitive function is Turing-computable (see Chapter 1, COMPUTATION).

Since not all functions are computable by a Turing machine, a way to try to falsify the computational theory of mind is by showing that there are cognitive functions that are not so computable. Whether there are such cognitive functions is a subject of dispute, and so this remains one line of attack on computationalism (Lucas 1961; Penrose 1989; van Gelder 1995; McCall 1999; Copeland 2000; see Lewis 1979 for a response to Lucas; and see Chalmers 1996 for responses to Penrose).

The Holy Grail of artificial intelligence

Since the functional architecture of our minds resides in our brains, Marvin Minsky has called our minds “meat machines.” The Holy Grail of the field of artificial intelligence (see Chapter 9, THE PHILOSOPHY OF AI AND ITS CRITIQUE) is to build something with mental abilities out of something other than living tissue. In pursuit of this Holy Grail, AI presupposes the computational theory of mind. If the computational theory of mind is correct, then it is at least logically possible for a mind essentially like ours to be made of quite different stuff from ours. The reason is that what is essential are the structures and their functional organization, not the material of which the elements of the structures are made. If the computational theory is right, then it is at least logically possible that the relevant structures with the relevant functional organization can be realized in something that is, for example, silicon based, rather than carbon based.

AI has been impressively successful in designing machines that can perform difficult tasks without our supervision. Moreover, it has been at the cutting edge of research into how cognitive functions might be computed. Finding the Holy Grail, however, remains an unrealized dream. We have computers that play master's-level chess and teams of robots that play soccer, but there are no artifacts that are even remotely plausible candidates for being the subjects of mental abilities. And despite the optimism of some AI researchers, there do not seem to be any in the works. It is

important to note, then, that the computational theory of mind is not committed to the success of this AI project. It leaves it an open empirical question whether the dream will ever be realized. For all we know, it may be that given the laws of nature and the initial conditions of the universe it is physically impossible for a mind (one essentially like ours) to be realized in something composed entirely of (e.g.) silicon. Such a mind might be impossible in the way that a 90-foot tall human being is impossible. The computational theory leaves this empirical issue open.

Nor is the computational theory of mind committed to Turing's (1950) would-be test (see Chapter 9) for determining whether something is genuinely intelligent (as opposed to merely appearing intelligent). The theory leaves open that there is more to intelligence than matching the dispositions to verbal or written behavior of an intelligent human being. It is not committed to the view that there is some pattern of dispositions to outward or peripheral behavior that suffices for the possession of genuine intelligence (Block 1981; Jackson 1993; McLaughlin & O'Leary-Hawthorne 1994).

2 The Symbol-system Paradigm

Mind as an interpreted automatic formal system

According to proponents of the symbol-system paradigm, the mind is an interpreted automatic formal system (Haugeland 1985). More precisely, having mental abilities consists in being constituted, at least partly, by an interpreted automatic formal system. A formal system is a system for manipulating discrete items in virtue of their formal properties; an automatic formal system is one that does so automatically. Many games, among them chess and checkers, are formal systems. Games that are formal systems share the following features: they are finitely playable, and are played by manipulating discrete items according to the rules of the game. In chess for instance, the discrete items are the chess pieces, which are manipulated according to the rules of chess. An automated formal system is one in

which discrete items are automatically manipulated according to the rules – like a chess set that plays chess by itself. An interpreted automatic formal system is an automated formal system in which the discrete items that are manipulated include symbols or representations: discrete items with semantic and formal properties. The rules by which they are manipulated prescribe algorithmic operations on them. Thus, this paradigm is sometimes called “the rules and representations” paradigm.

Turing machines are interpreted automatic formal systems. There are many computationally equivalent machines that are automatic formal systems with different functional architectures from that of a Turing machine (see Chapter 1, COMPUTATION). One such machine is a von Neumann machine, so named after John von Neumann. Virtually all commercial computers are von Neumann machines. The functional architecture of such a machine includes a Central Processing Unit (CPU), an arithmetic unit, memory locations, and two kinds of memory access. The CPU has access to memory locations and current active data structures and determines what operations the computer is to perform by consulting instructions (programs) located in memory. A somewhat similar functional architecture is implicit in some symbolic models of mental abilities: online processing is done using a short-term memory store that holds information relevant to the process being carried out; and online processing influences and is influenced by long-term memory. Nevertheless, it is universally accepted by computationalists that the functional architecture of the mind is not von Neumann architecture.

Higher-level programming languages such as Basic, Pascal, FORTRAN, Cobol, Java, and C++, and Lisp (List Processing) are (or describe) universal machines that are automatic formal systems. Such languages are very useful since, as John Haugeland (1985) has aptly noted, some machines are easy to build and some are easy to program. So we build the machines that are easy to build and use them to simulate the machines that are easy to program. Higher-level programming languages are thus simulated machines relative to the machine language of the computers we actually build; they are thus virtual machines

relative to the machine language of the computer. (A compiler is a program that translates the higher-level language into the machine language of the actual physical machine – in the case of von Neumann machines, into instructions encoded into strings of 1's and 0's.). No one thinks that any of these higher-level programming languages characterize the functional architecture of the mind.

Production Systems are Turing-machine equivalent higher-level programming languages used in research in the symbolic paradigm. Production Systems consist of rules specifying actions to be performed when certain conditions are met. While some computationalists think that at least some cognitive modules may have a Production System architecture, it is widely held by researchers in the symbolic paradigm that we do not yet know what the functional architecture of the mind is. The fundamental research objective of the symbolic paradigm is to determine what kind of automatic formal system the mind is.

Mind as syntactic engine

Recall that while a symbolic algorithm is defined over the formal properties of symbols (e.g., their shapes or syntactic structures), symbolic algorithms can govern symbol-to-symbol transitions that make sense given the meanings of the symbols. In proof theory, logical relations are treated as moves in a formal game; the formal moves preserve truth: they never take one from a truth to a falsehood. An algorithm can preserve truth: if one begins with a true symbol, the algorithm will take one only to a true symbol. The symbol-system paradigm is thus often said to presuppose a proof-theoretic conception of mind. Moreover, algorithms can be defined over the formal properties of symbols so as to preserve other sorts of semantic properties of the symbols, so that symbol transitions can implement reasoning processes of all sorts, not only deductive reasoning, but inductive reasoning, analogical reasoning, decision-making, etc. Haugeland (1985) suggests the following as the symbolist motto: "Take care of the syntax (the formal operations), and the semantics will take care of itself." As Daniel Dennett has aptly noted, the symbolic or rules

and representations paradigm presupposes that the mind is "a syntactic engine."

The language of thought

Jerry Fodor and Zenon Pylyshyn have articulated a research program in the symbolic paradigm for reductively explaining propositional attitudes and mental processes involving them, a program that invokes the hypothesis that there is "a language of thought" (Fodor 1975; Pylyshyn 1986; Fodor & Pylyshyn 1988). Propositional attitudes have an intentional mode and a propositional content. The intentional modes include belief, desire, hope, wish, fear, intention, and the like. The propositional contents of propositional attitudes are expressed using that-clauses. Thus, the belief that that the cat is on the mat has the propositional content that the cat is on the mat; and the hope that it will not rain has the propositional content that it will not rain. According to the language-of-thought hypothesis, some mental symbols are atomic, and some are molecular in that they contain other mental symbols as constituents. Moreover, the mental symbol system has a compositional semantics: the content or meaning of a molecular symbol is a function of its syntactic structure and the contents of its constituent atomic symbols. The contents of propositional attitudes are expressed by sentence-like mental symbols; the contents of concepts, by word-like symbols. On this view, to be in a state within a certain intentional mode is to be disposed to compute in a certain way with an amalgam of concepts with a sentence-like syntactic structure – a mentalese sentence. Thus, to believe that p is to be disposed to compute in a certain way with a mentalese sentence that means that p ; to desire that p is to be disposed to compute in a certain other way with a mentalese sentence that means that p , and so on for the other propositional attitudes. It is a topic for empirical investigation what the grammar of the language of thought is and what, exactly, these ways of being disposed to compute are.

As Fodor and Pylyshyn (1988) point out, if the mental symbol system includes a language of thought in the above sense, then we can appeal to it to explain, among other things, the systematicity and productivity of thought. They claim

that thought is productive in that one can, ideally, think an infinite number of thoughts; one is prevented from doing so only by the fact that one has a finite lifespan and finite memory resources. And they claim that thought is systematic in that pairs of abilities to have thoughts in the same intentional mode and with related contents are counterfactually dependent, in that one would have one member of the pair if one would have the other. Thus, normally, an individual that can think that Romeo loves Juliet can think that Juliet loves Romeo; and, normally, an individual that can hope the rock is next to the tree can hope the tree is next to the rock.

Psychosemantics

The question naturally arises of course as to how mental symbols – mental representations – acquire their contents or meanings. The numerals of various numeral systems such as Arabic numerals and Roman numerals derive their meanings from the linguistic conventions of communities. The current along the input and output wires of an electronic computer circuit derive their meanings from our intentional stipulative assignments; as do the keys of hand-calculators and the displays on their screen. If intentionality is to be explained by appeal to mental representations, then we need an account of their meaning that makes no appeal to linguistic conventions or semantic intentions, for these presuppose intentionality (Fodor 1990).

Procedural semantics and inferential or conceptual role semantics attempt to explain how mental representations derive their meaning from their participation in computational processes. John Searle's "Chinese Room" argument (see Chapter 9 above) makes a case that the semantic properties of symbols do not supervene on their syntactic relations, and so that intentional phenomena such as (e.g.) understanding Chinese cannot be explained solely by appeal to computational relations (see e.g. Searle 1999). If he is right about this failure of supervenience, then the kinds of semantics in question cannot succeed. Perhaps logical symbols – truth-functional symbols (e.g., "and" and "or") and symbols for the universal and existential quantifiers (respectively,

"all" and "some") – can be defined by patterns of inferential relations. However, it seems that most symbols (e.g., "cow" and "run") cannot.

Semantic externalists claim that the meanings of many mental symbols fail to supervene on anything that goes on in our heads, since they depend on environmental relations. Externalists typically claim that a computational theory of intentionality must thus be supplemented with an externalist naturalistic account of meaning that makes no appeal to intentional notions. There are various programs for attempting to explain the semantic properties of mental symbols in purely naturalistic terms. However, the naturalistic relations appealed to (e.g., causal relations, counterfactual dependencies, information-theoretic relations, processes of natural selection), all appear to leave the semantic properties of mental representations underdetermined. Dual-aspect semanticists hope to combine inferential or conceptual role semantics with a naturalistic externalist account to determine the semantic properties of mental representations (Block 1986). This project, however, faces the problem of specifying which inferential relations are constitutive of the internal component of meaning (Fodor & LePore 1992). Moreover, it is uncertain whether any such combination will yield determinate meanings for mental symbols. Saul Kripke (1981) has argued that since our inferential dispositions are in fact finite (due to limitations in memory, our finite lifespan, etc.), they will leave the meanings of symbols radically indeterminate. Thus, suppose that one is in fact disposed to perform only 500 million calculations with a symbol "+". Even if the output of each of these 500 million calculations can be systematically interpreted as a computation of the plus function, one's dispositions to calculate will be compatible with the symbols expressing instead the "quus function," which can be defined as follows: x quus $y = x$ plus y , for any $y < 500,000,001$, otherwise x quus $y = 5$. Conceptual role theorists take this problem to be one of how conceptual roles should be idealized. But no entirely satisfactory solution to the Kripke's problem has yet been proposed. Suffice it to note that the problem of how mental representations acquire their meanings is perhaps the deepest problem that any reductive theory of intentionality faces.

3 The Connectionist Computational Paradigm

The functional architecture of our minds is somehow realized in our brains. One of the few things we know about the brain that seems to have a bearing on mentality is that it is, *inter alia*, a system of neural networks. Neural networks are not well understood; indeed neurons themselves are not well understood. There are somewhere between 50 and 500 different kinds of cortical neurons, and these different kinds of neurons appear to perform specialized functions not yet understood (Churchland & Sejnowski 1993). Nonetheless, just as top-down considerations of what cognitive functions are being computed can guide research into how mental abilities are seated in the brain, bottom-up considerations of the workings of neural networks can guide research into what algorithms are used to compute the functions in question. They can serve as a guide to discovering the functional architecture of the mind.

While the operations of actual neural networks are not well understood, the connectionist computational paradigm explores functional architectures that are at least “neurally inspired.” Some connectionist networks, called “artificial neural networks,” are specifically designed to approximate various kinds of real neural networks in various respects: the units (or nodes) in artificial neural networks are analogous to neurons, the connections among units analogous to axons, and the weights or strengths of the connections analogous to synapses. These networks are extensively employed in the growing field of computational neuroscience (see Churchland & Sejnowski 1993). The connectionist networks employed to model the computation of cognitive functions typically make no attempt at neural realism. Nonetheless, they have at least “a neural flavor.”

Connectionist networks

A connectionist network is composed of interconnected units (or nodes). Individual units do all the information processing: there is thus no executive or CPU. There is, moreover, no

program stored in memory; the program is implicit in the pattern of connectivity exhibited by the units. Many units process information simultaneously, and so the network as a whole engages in parallel distributed processing (PDP) (see McClelland & Rumelhart et al. 1986; Rumelhart & McClelland et al. 1986).

Units have states of activation. Depending on the network architecture, a unit may have only two states of activation, “on” and “off,” three or more discrete states of activation, or continuous levels of activation, bounded or unbounded. Units process information by changing or retaining their state of activation in response to activation signals from their sending units.

The connections among units can be of various strengths or weights, which affect the extent to which the activity output of a sending unit influences the activation states of receiving units. Connections can, moreover, be excitatory or inhibitory. The connection from a unit U_i to a unit U_j is excitatory if the activation output of U_i tends to increase the level of activation of U_j , and inhibitory if it tends to decrease it. The causal influence exerted by a sending unit on a receiving unit thus depends on two intrinsic properties of their connection: its weight and whether it is excitatory or inhibitory. The notation “ w_{ij} ” is used to stand for a real number that indexes the connection from unit U_i to unit U_j by its weight and kind. The number is positive when the connection is excitatory, negative when it is inhibitory. The weight of the connection is the absolute value of w_{ij} . In many networks, the extent and kind of causal influence a unit U_i exerts on a unit U_j is indexed by the product of w_{ij} and the activation value of U_i .

Whether a unit changes or retains its state of activation is a function of three factors: the totality of input to the unit, the unit’s current activation state, and the unit’s bias (if any), which may be positive or negative. A unit thus computes an activation function that maps its total network activity input (and external input if any), its current activation state, and its bias (if any) to an activation state. The total network activity input to U_j is the sum of all of the causal influences from sending units: it is the sum of the product of each activation value from a sending unit and the real number indexing the weight

and kind of connection the sending unit bears to U_i . While the output function of a unit can be linear, it is typically a threshold function, and thus nonlinear. If a unit has a negative bias, it may send 0, the null signal, unless its activation value exceeds a certain threshold. If it has a positive bias, it may send a certain non-0 activation value unless its activation state value falls below a certain threshold. In Hopfield networks, units have a sigmoidal (S-shaped) response to net input: their output only increases by a given amount given an increase in net input; after that, they increase no further. But units can also have Gaussian (bell-shaped) output functions and other sorts of nonlinear ones as well.

The input units of a network receive signals directly from the environment of the network, while the output units send signals directly to the environment. Since input and output units directly interact with the environment, they are called “visible units.” So-called “hidden units” only directly interact with other units.

Some networks, called “perceptrons,” have only two layers of units: a layer of input units and a layer of output units. Minsky and Pappert (1969) showed perceptrons are limited in their computational power: for example, they cannot compute XOR (exclusive-or). The reason is that the problem of determining the value of the XOR function is a linearly inseparable problem and perceptrons cannot solve any such problem. Rumelhart et al. 1986 demonstrated, however, that networks with three or more layers can compute XOR and, more generally, can solve decidable linearly inseparable problems.

Feedforward networks with one or more layers of hidden units are called “multilayered” networks. The Hamming net, for instance, is a widely used feedforward network with three layers of units, one of which is a layer of hidden units. Feedforward networks are non-interactive: activation flows from the input units through each layer of hidden units to the output units. In interactive networks, two units can mutually influence each other, and thus a unit can be related to another both as an input unit and as an output unit. In interactive networks, two-way connections between units are often symmetrical, so that $w_{ij} = w_{ji}$. In competitive networks, units form pools: the units in a pool are all mutually

inhibitory, while units outside of the pool bear excitatory connections to one or more units in the pool. In recurrent networks, there are connection patterns that contain loops, so that a unit is either related to itself as an input unit or there is a series of connections from the unit back to itself, so that the output of a unit at one time can causally influence its activation state at another. In auto-associative networks, each unit is connected to every other unit, including itself. These are only some of the many kinds of patterns of connectivity that are possible.

The behavior of a network as a whole is a consequence of the pattern of connectivity exhibited by its units at a time and the global activation state of the network at that time. A vector (ordered list) of real numbers is used to index the global activation state. The activation value of each unit in the network at that time is indexed by one and only one element of the vector, and so the number of elements in the vector will be equal to the number of units in the network. The set of all possible global activation states of a network is its activation space, whose dimensionality is exactly equal to the number of units in the network. A network with n units will thus be indexed by an n -tuple of real numbers that identifies a position in an n -dimensional vector space. That position represents the global activation state of the network at a time. A temporal series of global activation states will trace a path through a vector space. Network information processing is characterized as the evolution through time of global patterns of activation. Networks can be systematically interpreted as performing mathematical operations on matrices such as matrix multiplication (e.g., a computing inner product).

Explicit representations in a connectionist network can be either local or distributed. Local representations are individual units, or individual units at certain levels of activation. Distributed representations are patterns of activity over a group of units. The pattern of connectivity of a network is sometimes characterized as implicitly representing.

As a result of the pattern of connectivity exhibited by its units, a network as a whole can behave in rule-like ways to compute functions. When a feedforward network computes a

function, the arguments of the function are represented by different patterns of activity over the input units, and the values of those arguments for the function by patterns of activity over output units. In computing the function, the network acts as a look-up table. Input activation patterns function like questions posed to the network (for example, “To what category does this belong?”), and output patterns function like answers to the question (“To category C”). Unlike in a symbolic look-up table, however, the answers are not stored as data structures; rather they are implicitly represented in the pattern of connectivity. (Sometimes, however, hidden units are explicit representations.) In interactive units, the argument of a function might be represented by an initial pattern of activation over units in the network, and the value of the function for that argument by a pattern of activation over those same units, a pattern the network “settles” or “relaxes” into after information processing.

The “neural flavor” of connectionist networks is by no means their only attraction for computationalists. Connectionist networks are good at pattern recognition tasks: pattern matching, pattern completion, and pattern association. They degrade gracefully in their performance of a task as a network is damaged (e.g., when a unit is lost) and in the face of noisy or incomplete data. They can learn. That is to say, with proper training, they can improve their performance of various tasks. Moreover, they are very efficient at solving connected problems and at arriving at optimal or near optimal solutions to best-match problems. Of these attractions, more below.

Learning

The weights of the connections in a network are not architecturally fixed. Learning consists in changes in the weights. There are various learning rules that govern weight change. The Hebb Rule is based on Donald Hebb’s (1949) hypothesis that the connections between two neurons might strengthen whenever they fire simultaneously. According to the Hebb rule, the weight of a connection between units should be increased or decreased in proportion to the products of

their simultaneous activations (Rumelhart et al. 1986). The Delta rule is an error correction rule that changes the weights leading from units sending signals to output units on the basis of the discrepancy between the actual output and the desired one. Backpropagation is a generalization of the Delta rule, and is widely used in multi-layered networks. The actual output activation pattern for a given input activation pattern is compared with the desired output. The difference between the two is then propagated back into whatever connections were used to get the actual output activation pattern. The connections among units that contributed to the actual output are strengthened (increased in weight) when the match is good, and are reduced in strength (decreased in weight) when it is poor. The weights are thus adjusted so as to reduce the margin of error between the actual output and the desired one. And in this way, the network learns.

Network training can be supervised or unsupervised. In supervised learning, the network is provided explicit feedback from an external source about what output is desired as a response to a certain input. The Delta rule and backpropagation are used in supervised learning. In unsupervised learning, no such external feedback is provided to the network; rather, the network monitors its own performance through internal feedback. There are a number of unsupervised learning algorithms; the Kohonen algorithm is one such (see Beale & Jackson 1990, ch. 5).

Pattern recognition

NETtalk offers a vivid example of the ability of networks to learn to recognize patterns. Designed by Terrence Sejnowski and C. R. Rosenberg (1987), it is a network that learns to map letters onto phonemes. NETtalk is a three-layered feedforward network, whose input units represent letters (individual letters are represented by patterns of activation over 29 input units and there are 7 such groups of 29 input units) and whose output units represent phonemes. The network feeds into a synthesizer. After sufficient training using backpropagation, when presented strings of letters comprising the words of actual

English text, the network drives the synthesizer to sound like a robotic voice literally reading the text.

Some studies have compared networks using backpropagation with various members of the class of “top-down inductive decision trees” (“decision trees” for short) in respect of accuracy in pattern recognition and the ability to deal with noisy, incomplete data gracefully, as well as in other respects (see Sethi & Jain 1991). Decision trees are production-rule systems, and thus symbolic systems, that excel at pattern recognition tasks (see Utgoff 1999). One comparative study by Shavlik et al. (1991) included a comparison between a decision tree system called “ID3” and a network using backpropagation on both the NETtalk A data set and the NETtalk full data set. Shavlik et al. noted: “for the most part [both] learning systems are similar with respect to accurately classifying novel examples” (1991: 120). They also compared ID3 and the network on three kinds of noisy data: inaccurate feature values, missing feature values, and insufficient number of features. The network performed slightly better than ID3 on inaccurate feature values and missing feature values, but when trained on insufficient numbers of feature values, “ID3 and backpropagation degrade at roughly the same rate as the number of features is reduced” (1991: 130). Several other comparative studies have found that networks using backpropagation are roughly comparable to various other members of the family of decision trees as concerns accuracy in pattern recognition and graceful degradation in response to noisy, incomplete data. Decision trees are, however, much faster at learning than networks using backpropagation (see Sethi & Jain 1991; Marinov 1993; and McLaughlin & Warfield 1994).

*Connected problems, best-match
problems, and multiple soft constraint
satisfaction*

Networks are especially efficient at solving connected problems – problems that do not divide into independently solvable subproblems, like the traveling salesman problem. The goal is to find the shortest route that a salesman can take to

visit each of a number of cities, while visiting each city only once. Since which city the salesman visits depends on which cities he has already visited, the problem does not divide into independently solvable subproblems. The traveling salesman problem is decidable, and so can be solved by a symbol system. But if, for instance, there are 20 cities to visit, there is a minimum of 653,837,184,000 possible routes to take (Raggett & Bains 1992). Thus, as the number of cities increases there is an exponential increase in the computational resources required to solve the traveling salesman problem within a symbol system (see Chapter 2, COMPLEXITY). A Hopfield network is able to find a solution in a small number of training cycles.

Networks are also especially efficient at finding optimal or near optimal solutions to what Minsky and Papert (1969) called “best-match problems” – problems whose solution involves assessing the satisfaction of multiple soft (i.e., non-mandatory) constraints. Hinton (1977) developed the first network approach to solving best-match problems. The following description of it paraphrases the description provided in McClelland and Rumelhart 1988. Units in the network have one of two states of activation (“on” or “off”), and the network contains local representations, each unit representing a different hypothesis. The connections in the network implicitly represent evidential relationships among the hypotheses. Thus if a hypothesis H is evidence for another hypothesis H' , the connection from the unit representing H to the unit representing H' is positive; if H is evidence against H' , the connection is negative. The stronger the confirming or disconfirming relationship the one hypothesis bears to the other, the stronger the connection between the units that represent them. If one hypothesis H entails another H' , then the connection between the unit representing H and the unit representing H' will be such that the second unit is on whenever the first is on. When two hypotheses are incompatible, they are connected in such a way that they cannot both be on. Since the network contains input units that receive signals from the environment, hypotheses can receive confirming or disconfirming evidence directly from the environment. The fact that different hypotheses have different *a priori*

probabilities is captured by biasing the relevant units. The overall goodness of fit of a particular hypothesis to evidence is measured by the sum of the individual constraints on the activation value of the unit representing the hypothesis. The activation values of units range between a minimum and a maximum. The maximum value means that the hypothesis should be accepted, the minimum that it should be rejected, and intermediate values correspond to intermediate levels of certainty. The constraint satisfaction problem is thus reduced to one of maximizing this overall goodness of constraint fit.

Of course, this approach to best-match problems may well not seem interestingly different from a symbolic approach; indeed it might be viewed as a way of implementing a symbolic decision procedure on a network. The computationally relevant formal property of a symbol can be a unit's being activated, and symbols can participate in probabilistic algorithmic processes. There are other networks, however, that solve best-match problems in strikingly different, and striking efficient ways.

A problem that arises for procedures for solving best-match problems is that of avoiding local maxima of goodness of constraint fit. It can be characterized as an energy minimization problem (McClelland & Rumelhart 1988). The analog of the goodness maximum is the energy minimum, and the analogs of local goodness maxima are local energy minima. The situation is easy to visualize as an energy landscape. In an energy landscape, the goodness maximum corresponds to the lowest valley in the landscape, while local goodness minima correspond to local valleys in the landscape. The problem of avoiding local goodness maxima is thus the problem of avoiding settling into a local valley, rather than into the lowest valley in the energy landscape. John Hopfield (1982) observed that some networks behave in such a way as to minimize a measure over the whole network, one he aptly called "energy." The behavior of these networks can be characterized as moving through an energy landscape. The Boltzmann machine, developed by Hinton and Sejnowski (1986) and named after the physicist Ludwig Boltzmann, is such a network. To handle the problem of local maxima of goodness of constraint fit, the Boltzmann

machine employs a computational analog of the metallurgical process of annealing. Annealing is a process whereby metals are heated to a little below their melting point and then cooled very slowly so that all their atoms have time to settle into a single orientation. The analog of heat in the Boltzmann machine is random noise that is introduced into network activity. The function of the noise is to jar the network out of local valleys, so that it can explore other parts of the energy landscape to find the lowest valley, thereby achieving the global maximum of fit. When the network reaches a stable state, it has settled or relaxed into a solution. Given sufficient time, the Boltzmann machine can find the energy minimum for any best-match problem.

Networks and cognitive abilities

There are network models of certain aspects of motor control and low-level perception. For example, there is a network model for the vestibular ocular reflex, which enables eyes to track an object as the head moves (Churchland & Sejnowski 1993). As concerns low-level visual perception, the linear models color-constancy algorithm, for instance, is characterized in connectionist terms (Maloney & Wandell 1986). Moreover, opponent processing theory, the leading neuro-computational theory of color experience, is easily understood in connectionist terms (Hurvich 1981). According to the theory, there are pairs of opponent channels that respond differently to the outputs of our three types of cones (L-cones, M-cones, and S-cones): the RED channel and the GREEN channel, the BLUE channel and the YELLOW channel, and the BLACK channel and the WHITE channel. Activity in one channel inhibits activity in its opponent channel. The theory explains the fact that nothing looks bluish-yellow as a result of the fact that activity in the BLUE channel inhibits activity in the YELLOW channel and conversely. And the theory explains the appearance of unique blue (blue that is not at all reddish or greenish) as the result of the activation of the BLUE channel, the YELLOW channel being deactivated, and the RED and GREEN channels being in equilibrium. These channels can, of course, be

understood as pools of interconnected neuron-like units.

Connectionist modeling has by no means been restricted to motor control and low-level perception. It has been extended to most areas of cognition. Connectionist networks have been appealed to as mechanisms for processes of categorization in terms of resemblance to prototypes (Churchland 1995). And there are, moreover, connectionist models of various aspects of language comprehension and production. For example, Jeffrey Elman (1990) has explored how recurrent nets can learn to become sensitive to anaphoric relations. And Alan Prince and Paul Smolensky's (1993) optimality theory, for instance, outlines how a multiple soft constraint satisfaction might serve as a natural language parser might work by employing multiple soft constraint satisfaction. This sample represents only a tiny fraction of connectionist cognitive modeling. (For a recent very brief overview, see McClelland 1999.)

4 How are Paradigms Related?

The relationship between the symbolic and connectionist paradigms is a topic of heated dispute. It will have to suffice here simply to list some possibilities.

One possibility is that the functional architecture of the mind is hybrid: some aspects of cognition are symbolic and some are connectionist. For example, perhaps motor control modules and low-level perceptual modules have a connectionist architecture, while linguistic modules and reasoning in central processing have a symbolic architecture. On this view, the mind is not a single kind of computer, but has different kinds of compartments that are different kinds of computers.

Another possibility is that the functional architecture of the mind is a symbolic architecture, but this architecture is implemented by a connectionist one. (Both Turing machines and Production Systems, it should be mentioned, have been implemented by connectionist networks.) Connectionism, it is often claimed, is concerned with microcognition; and its advocates often characterize the connectionist paradigm as

the subsymbolic paradigm. Some connectionists have suggested that the relationship between the symbolic paradigm and the connectionist (subsymbolic) paradigm is analogous to the relationship that Newtonian physics bears to quantum mechanics (Rumelhart et al. 1986). But proponents of the implementational view of connectionism will claim that the more apt analogy is the relationship chemistry bears to quantum mechanics. Atoms are constituted by subatomic particles, and chemical processes are implemented by quantum mechanical ones (indeed all causal processes are ultimately implemented by quantum mechanical ones). Perhaps, analogously, atomic symbols are constituted by patterns of activation, and connectionist processes implement symbolic algorithmic processes (McLaughlin 1993a).

Yet another possibility is that the functional architecture of the mind is either thoroughly symbolic or thoroughly connectionist. Proponents of this view see the two paradigms as in a sort of zero-sum competition. The leading objection to the view that the functional architecture of the mind is thoroughly connectionist is due to Fodor and Pylyshyn (1988). They argue that a connectionist architecture cannot explain the systematicity of thought without implementing a symbolic architecture. A large literature has arisen in response to this objection. Some deny that thought is systematic. Some concede that thought is systematic, but maintain that it is not the job of a theory of cognitive architecture to explain systematicity (see McLaughlin 1993b for a discussion). And some attempt to show how a connectionist architecture could explain the systematicity of thought without implementing a symbolic one (see e.g. Smolensky 1991; for a response, see Fodor & McLaughlin 1990; McLaughlin 1997a; for a counter-response, see Cummins 2000).

Another possibility still is that the functional architecture of the mind somehow integrates features of symbolic architectures and features of connectionist architectures. Smolensky (1994, 1995) has sketched an architecture he calls "an Integrated Connectionist Symbol architecture (ICS)" that attempts to include features of both connectionist and symbol architectures, and which he claims will explain the systematicity and

productivity of thought. But it remains a question whether when developed sufficiently to explain systematicity and productivity, it would collapse into an implementation architecture for a symbolic one (McLaughlin 1997a).

Of course, not all of these possibilities are exclusive. It may be that some cognitive modules are thoroughly connectionist, and that various other modules are symbolic but implemented by connectionist networks. There is, it should be noted, a growing body of work in the field of machine learning that attempts to develop machines with a mixed architecture, including symbolic and connectionist subcomponents. Of course, this work in machine learning is not concerned with either psychological or neurological realism (but rather with building better machines). Nonetheless, it demonstrates the usefulness of such mixed architectures for computational purposes.

Suffice it to note that the functional architecture of the mind remains an open question. What the future will bring remains to be seen.

References

- Beale, R. and Jackson, T. 1990. *Neural Computing*. Bristol: Adam Hilger. [An excellent introduction to neural network computing that is accessible to advanced undergraduates.]
- Bechtel, W. and Abrahamsen, A. 1991. *Connectionism and the Mind*. Oxford: Blackwell. [An excellent introduction to connectionist networks and their contribution to the understanding of mentality, and a major source of the discussion of the connectionist paradigm in this chapter. Accessible to advanced undergraduates.]
- Block, N. 1981. "Psychologism and behaviorism." *Philosophical Review* 90: 5–43. [A defense of the view that mentality does not supervene on any pattern of dispositions to outward or peripheral behavior. Accessible to advanced undergraduates.]
- . 1986. "An advertisement for a semantics for psychology." *Midwest Studies in Philosophy* 10: 615–78. [A seminal defense of conceptual role semantics. Accessible to advanced undergraduates.]
- Chalmers, D., ed. 1996. *Psyche*, vol. 2, <<http://Psyche.cs.monash.edu.au/>>. [Contains responses to Penrose 1989. Accessible to graduate students and professionals.]
- Churchland, P. M. 1995. *The Engine of Reason, the Seat of the Soul: A Philosophical Journey Into the Brain*. Cambridge, MA: MIT/Bradford. [Discussion of the role of networks in understanding mentality. Accessible to advance undergraduates.]
- Churchland, P. S. and Sejnowski, T. J. 1993. *The Computational Brain*. Cambridge, MA: Bradford/MIT. [An introduction to neural network modeling in the field of computational neuroscience. Accessible to advanced undergraduates.]
- Copeland, J. B. 2000. "Narrow versus wide mechanism: including a re-examination of Turing's views on the mind-machine issue." *Journal of Philosophy* 1: 5–32. [A defense of the view that a mechanistic view of the mind need not treat minds as Turing equivalent computational mechanisms. Accessible to advanced undergraduates.]
- Cummins, R. and Schwarz, G. 1991. "Connectionism, computation, and cognition." In Horgan & Tienson 1991: 60–73. [A discussion of the role of connectionism in the computational theory of mind. Accessible to advanced undergraduates.]
- Elman, J. L. 1990. "Finding structure in time." *Cognitive Science* 14: 179–211. [An attempt to show how connectionist networks can learn to compute certain linguistic functions. Accessible to graduate students.]
- Fodor, J. 1975. *The Language of Thought*. New York: Crowell. [A seminal defense of the language-of-thought hypothesis. Accessible to advanced undergraduates.]
- . 1990. *A Theory of Content and Other Essays*. Cambridge, MA: MIT/Bradford. [A seminal discussion of issues in psychosemantics. Accessible to advanced undergraduates.]
- and LePore, E. 1992. *Holism*. Malden, MA: Blackwell. [Contains criticisms of inferential/conceptual role semantics. Accessible to advanced undergraduates.]
- and McLaughlin, B. P. 1990. "Connectionism and the problem of systematicity: why Smolensky's solution doesn't work." *Cognition* 35: 183–204. Repr. in Horgan & Tienson 1991. [A response to Smolensky 1991. Accessible to advanced undergraduates.]
- and Pylyshyn, Z. 1988. "Connectionism and cognitive architecture." *Cognition* 28: 183–204. [A classical attack on the view that cognitive architecture may be thoroughly connectionist. Accessible to advanced undergraduates.]
- Haugeland, J. 1985. *Artificial Intelligence: The Very Idea*. Cambridge, MA: MIT/Bradford.

- [An excellent introduction to the symbolic approach to artificial intelligence. Accessible to undergraduates.]
- , ed. 1997. *Mind Design II*. Cambridge, MA: MIT/Bradford. [Contains classical articles in the field of artificial intelligence and the computational theory of mind. Accessible to advanced undergraduates.]
- Hebb, D. 1949. *The Organization of Behavior*. New York: Wiley & Sons. [A classical work on neural networks. Accessible to advanced undergraduates.]
- Hinton, G. E. and Sejnowski, T. J. 1986. "Learning and relearning in Boltzmann machines." In Rumelhart et al. 1986: 282–317. [A presentation of Boltzmann machines. Accessible to graduate students.]
- Hopfield, J. J. 1982. "Neural networks and physical systems with emergent collective computational abilities." *Proceedings of the National Academy of Sciences, USA*, 79: 2554–8. [Presents what have come to be called "Hopfield nets." Accessible to graduate students.]
- Horgan, T. and Tienson, J., eds. 1991. *Connectionism and the Philosophy of Mind*. Dordrecht: Kluwer. [A collection of essays on philosophical issues concerning connectionism. Accessible to advanced undergraduates.]
- Hurvich, L. M. 1981. *Color Vision*. Sunderland, MA: Sinauer.
- Jackson, J. 1993. "Appendix A [for philosophers]." *Philosophy and Phenomenological Research* 53: 901–6. [Argues that mentality does not supervene on dispositions to outward or peripheral behavior. Accessible to advanced undergraduates.]
- Kripke, S. 1982. *Wittgenstein on Rules and Private Language*. Cambridge, MA: Harvard University Press. [Contains a presentation of the plus-quus problem for computationalists. Accessible to advanced undergraduates.]
- Lewis, D. 1979. "Lucas against mechanism II." *Canadian Journal of Philosophy* 9: 373–6. [Response to Lucas 1961. Accessible to graduate students and professionals.]
- Lucas, J. R. 1961. "Minds, machines and Gödel." *Journal of Philosophy* 36: 112–27. [Appeals to the results of Kurt Gödel to argue that we are not computationally equivalent to Turing machines. Accessible to graduate students.]
- Macdonald, C. and Macdonald, G., eds. 1995. *Connectionism: Debates on Psychological Explanation*. Malden, MA: Blackwell. [A collection of articles on philosophical issues surrounding the connectionist approach to the mind. Accessible to advanced undergraduates.]
- Maloney, L. T. and Wandell, B. A. 1986. "Color constancy: a method for recovering surface spectral reflectance." *Journal of the Optical Society of America A* 3: 29–33. [Presents the linear models color-constancy algorithm. Accessible to graduate students.]
- Marinov, M. S. 1993. "On the spuriousness of the symbolic/subsymbolic distinction." *Minds and Machines* 3: 253–70. [Contains a discussion of comparative studies of connectionist networks and decision trees. Accessible to advanced undergraduates.]
- Marr, D. 1982. *Vision*. New York: Freeman. [A seminal work on low-level vision. Accessible to graduate students.]
- McCall, S. 1999. "Can a Turing machine know that the Gödel sentence is true?" *Journal of Philosophy* 10: 525–32. [Appeals to results of Kurt Gödel to argue that we are not computationally equivalent to Turing machines. Accessible to graduate students.]
- McClelland, J. L. 1999. "Cognitive modeling, connectionist." In Wilson & Kiel 1999: 137–41. [A brief overview of connectionist modeling. Accessible to advanced undergraduates.]
- and Rumelhart, D. E. 1988. *Explorations in Parallel Distributed Processing: A Handbook of Models, Programs, and Exercises*. Cambridge, MA: MIT/Bradford. [An introduction to connectionist modeling, and a major source of the discussion of the connectionist paradigm in this chapter. Accessible to advanced undergraduates.]
- , ——, and the PDP Group, eds. 1986. *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*, vol. 2, *Psychological and Biological Models*. Cambridge, MA: MIT/Bradford. [One of the two "bibles" of connectionism. This seminal collection of essays that introduces and develops the connectionist paradigm. Some of the chapters are accessible to advanced undergraduates, but some are more appropriate for graduate students and professionals.]
- McLaughlin, B. P. 1993a. "The connectionism/classicism battle to win souls." *Philosophical Studies* 70: 45–72. [An examination of the debate between the symbolic paradigm and the connectionist paradigm. Accessible to advanced undergraduates.]
- . 1993b. "Systematicity, conceptual truth, and evolution." In C. Hookway and D. Peterson, eds., *Royal Institute of Philosophy, Supplement* 34:

- 217–34. [Examines some attempts to show that it is not the job of the computational theory of mind to explain the systematicity of thought. Accessible to advanced undergraduates.]
- . 1997a. “Classical constituents in Smolensky’s ICS architecture.” In M. L. Dalla Chiara, K. Doets, D. Mundici, and J. van Bentham, eds., *The Tenth International Congress of Logic, Methodology and Philosophy of Science, Florence, August 1995*, vol. 2, *Structures in Science*. Dordrecht: Kluwer, 331–43. [Argues that Smolensky’s ICS architecture can contain mental representations with classical constituents. Accessible to advanced undergraduates.]
- . 1997b. “Connectionism.” In E. Craig, ed., *The Routledge Encyclopedia of Philosophy*. London: Routledge. [An introduction to the connectionist paradigm. A source for the discussion of the connectionist paradigm in this chapter.]
- and O’Leary-Hawthorne, J. 1994. “Dennett’s logical behaviorism.” *Philosophical Topics* 22: 189–258. [Presents Daniel Dennett’s intentional systems theory, and argues that, *contra* that theory, intentionality does not supervene on any pattern of dispositions to outward or peripheral behavior. Accessible to advanced undergraduates.]
- and Warfield, T. A. 1994. “The allure of connectionism reexamined.” *Synthese* 101: 365–400. [Contains a discussion of studies comparing connectionist networks and decision trees. Accessible to advanced undergraduates.]
- Minsky, M. A. and Pappert, S. 1969. *Perceptrons*. Cambridge, MA: MIT Press. [A critical examination of perceptrons, two-layered connectionist networks. Accessible to graduate students.]
- Penrose, R. 1989. *The Emperor’s New Mind: Concerning Computers, Minds, and the Laws of Physics*. New York: Oxford University Press. [Includes an appeal to results of Kurt Gödel to argue that our minds are not computationally equivalent to Turing machines. Accessible to graduate students.]
- Prince, A. and Smolensky, P. 1993. *Optimality Theory: Constraint and Interaction in Generative Grammar*. Technical report TR-2, New Brunswick, NJ: Rutgers Center for Cognitive Sciences. [A seminal presentation of optimality theory. Accessible to graduate students.]
- Putnam, H. 1975. “Minds and machines.” In his *Philosophical Papers*, vol. 2, *Mind, Language, and Reality*. Cambridge: Cambridge University Press. [A classical paper on machine state psychofunctionalism. Accessible to advanced undergraduates.]
- Pylshyn, Z. 1986. *Computation and Cognition*. Cambridge, MA: MIT/Bradford. [A seminal discussion of the symbolic paradigm, and a major source for the discussion of that paradigm in this chapter. Accessible to advanced undergraduates.]
- Raggett, J. and Bains, W. 1992. *Artificial Intelligence From A to Z*. London: Chapman & Hall. [A wonderful source of very simple explanations of technical terms in the field of artificial intelligence. Accessible to advanced undergraduates.]
- Ramsey, W., Stich, S., and Rumelhart, D., eds. 1991. *Philosophy and Connectionism Theory*. Hillsdale, NJ: Erlbaum. [A collection of articles on the connectionist paradigm. Accessible to advanced undergraduates.]
- Rumelhart, D. E., McClelland, J. L., and the PDP Research Group, eds. 1986. *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*, vol. 1, *Foundations*. Cambridge, MA: MIT/Bradford. [One of the two “bibles” of connectionism. A classical introduction of the connectionist paradigm, and a major source for the discussion of that paradigm in this chapter. Some chapters are accessible to advanced undergraduates, but some are most appropriate for graduate students and professionals.]
- Searle, J. 1999. “Chinese room argument.” In Wilson & Keil 1999: 115–16. [Searle’s most recent presentation of his well-known Chinese room argument. Accessible to undergraduates.]
- Sejnowski, J. and Rosenberg, C. R. 1987. “Parallel networks that learn to pronounce English text.” *Complex Systems* 1: 145–68. [Introduces NETalk. Accessible to graduate students.]
- Sethi, I. K. and Jain, A. K., eds. 1991. *Artificial Neural Networks and Statistical Pattern Recognition*. Amsterdam: Elsevier/North Holland. [A collection of essays including some comparative studies of connectionist networks and decision trees. Accessible to graduate students.]
- Shavlik, J. W., Mooney, R. J., and Towell, G. G. 1991. “Symbolic and neural learning algorithms: an experimental comparison.” *Machine Learning* 6: 111–44.
- Smolensky, P. 1991. “The constituent structure of connectionist mental states: a reply to Fodor and Pylshyn.” In Horgan & Tienson 1991: 281–308. [A reply to Fodor & Pylshyn 1988. Accessible to advanced undergraduates.]
- . 1994. “Computational models of mind.” In Guttenplan 1994: 176–84. [An introduction to the connectionist paradigm that includes a sketch

- of Smolensky's ICS architecture. Accessible to advanced undergraduates.]
- . 1995. "Reply: constitutive structure and explanation in an integrated connectionist/symbolic cognitive architecture." In Macdonald & Macdonald 1995: 223–90.
- Sterelny, K. 1990. *The Representational Theory of Mind: An Introduction*. Oxford: Blackwell. [An excellent introduction to the representational theory of mind. Accessible to advanced undergraduates.]
- Turing, A. 1950. "Computing machinery and intelligence." *Mind* 59: 433–60. [A truly groundbreaking article that literally began the field of artificial intelligence. Accessible to undergraduates.]
- Utgoff, P. 1999. "Decision trees." In Wilson & Kiel 1999: 223–5. [An introduction to decision trees. Accessible to advanced undergraduates.]
- Van Gelder, T. 1995. "What might cognition be, if not computation?" *Journal of Philosophy* 7: 345–81. [A defense of the view that cognition should be understood in dynamical systems terms, rather than computationally. Accessible to advanced undergraduates.]
- Wilson, R. A. and Keil, F. C., eds. 1999. *The MIT Encyclopedia of the Cognitive Sciences*. Cambridge, MA: MIT/Bradford. [Contains many entries relevant to topics discussed in this article. Entries are accessible to advanced undergraduates.]